

Module 2: Essentials of Python

Author: Lei Wu

Date: 2023-01-01

Version: 1.0.0

Learning Objectives

- Advanced Python data structures
- Object-oriented programming concepts
- File handling and I/O operations
- Exception handling
- List comprehensions and advanced features

Topics Covered

1. Lists, tuples, and dictionaries
2. Sets and their operations
3. String manipulation and formatting
4. List comprehensions and generator expressions
5. Object-oriented programming basics
6. Classes and objects
7. Inheritance and polymorphism
8. File reading and writing
9. Exception handling (try/except/finally)
10. Regular expressions
11. Working with dates and time

Exercises

- Hands-on coding exercises for each topic

1. Lists, Tuples, and Dictionaries

In [1...

```
1
2  # Lists - mutable ordered collection
3  fruits = ["apple", "banana", "orange"]
4  print(f"Original list: {fruits}")
5
6  # List operations
7  fruits.append("grape")
8  print(f"After append: {fruits}")
9
10 fruits.insert(1, "mango")
```

```

11     print(f"After insert: {fruits}")
12
13     fruits.remove("banana")
14     print(f"After remove: {fruits}")
15
16     # List slicing
17     print(f"First two items: {fruits[:2]}")
18     print(f>Last two items: {fruits[-2:]}")
19
20     # Tuples - immutable ordered collection
21     coordinates = (10, 20)
22     print(f"Coordinates: {coordinates}")
23     print(f"X coordinate: {coordinates[0]}")
24
25     # Tuple unpacking
26     x, y = coordinates
27     print(f"X: {x}, Y: {y}")
28
29     # Dictionaries - key-value pairs
30     person = {
31         "name": "Alice",
32         "age": 30,
33         "city": "New York"
34     }
35     print(f"Person: {person}")
36     print(f"Name: {person['name']}")
37     print(f"Age: {person.get('age', 'Unknown')}")
38
39     # Dictionary operations
40     person["occupation"] = "Engineer"
41     print(f"After adding occupation: {person}")
42
43     # Iterating through dictionary
44     for key, value in person.items():
45         print(f"{key}: {value}")

```

Original list: ['apple', 'banana', 'orange']

After append: ['apple', 'banana', 'orange', 'grape']

After insert: ['apple', 'mango', 'banana', 'orange', 'grape']

After remove: ['apple', 'mango', 'orange', 'grape']

First two items: ['apple', 'mango']

Last two items: ['orange', 'grape']

Coordinates: (10, 20)

X coordinate: 10

X: 10, Y: 20

Person: {'name': 'Alice', 'age': 30, 'city': 'New York'}

Name: Alice

Age: 30

After adding occupation: {'name': 'Alice', 'age': 30, 'city': 'New York', 'occupation': 'Engineer'}

name: Alice

age: 30

city: New York

occupation: Engineer

2. Sets and Their Operations

In [2...

```
1  # Creating sets
2  fruits = {"apple", "banana", "orange", "apple"} # Duplicates removed
3  print(f"Fruits set: {fruits}")
4
5
6  numbers = set([1, 2, 3, 4, 5, 2, 3]) # From list
7  print(f"Numbers set: {numbers}")
8
9  # Set operations
10 set1 = {1, 2, 3, 4, 5}
11 set2 = {4, 5, 6, 7, 8}
12
13 print(f"Set 1: {set1}")
14 print(f"Set 2: {set2}")
15
16 # Union
17 union = set1 | set2
18 print(f"Union: {union}")
19
20 # Intersection
21 intersection = set1 & set2
22 print(f"Intersection: {intersection}")
23
24 # Difference
25 difference = set1 - set2
26 print(f"Set1 - Set2: {difference}")
27
28 # Symmetric difference
29 symmetric_diff = set1 ^ set2
30 print(f"Symmetric difference: {symmetric_diff}")
31
32 # Set methods
33 set1.add(6)
34 print(f"After adding 6: {set1}")
35
36 set1.remove(1)
37 print(f"After removing 1: {set1}")
38
39 # Check membership
40 print(f"Is 3 in set1? {3 in set1}")
41 print(f"Is 10 in set1? {10 in set1}")
```

Fruits set: {'apple', 'orange', 'banana'}

Numbers set: {1, 2, 3, 4, 5}

Set 1: {1, 2, 3, 4, 5}

Set 2: {4, 5, 6, 7, 8}

Union: {1, 2, 3, 4, 5, 6, 7, 8}

Intersection: {4, 5}

Set1 - Set2: {1, 2, 3}

Symmetric difference: {1, 2, 3, 6, 7, 8}

After adding 6: {1, 2, 3, 4, 5, 6}

After removing 1: {2, 3, 4, 5, 6}

Is 3 in set1? True

Is 10 in set1? False

3. String Manipulation and Formatting

```
1  # Basic string operations
2  text = "Hello, World!"
3  print(f"Original: {text}")
4  print(f"Uppercase: {text.upper()}")
5  print(f"Lowercase: {text.lower()}")
6  print(f"Title case: {text.title()}")
7  print(f"Length: {len(text)}")
8
9
10 # String methods
11 sentence = " Python is awesome! "
12 print(f"Original: '{sentence}'")
13 print(f"Stripped: '{sentence.strip()}'")
14 print(f"Left stripped: '{sentence.lstrip()}'")
15 print(f"Right stripped: '{sentence.rstrip()}'")
16
17 # String splitting and joining
18 words = "apple,banana,orange,grape"
19 fruit_list = words.split(",")
20 print(f"Split by comma: {fruit_list}")
21
22 joined = " | ".join(fruit_list)
23 print(f"Joined with ' | ': {joined}")
24
25 # String formatting methods
26 name = "Alice"
27 age = 30
28
29 # f-string (Python 3.6+)
30 message1 = f"Hello, {name}! You are {age} years old."
31 print(f"f-string: {message1}")
32
33 # .format() method
34 message2 = "Hello, {}! You are {} years old.".format(name, age)
35 print(f".format(): {message2}")
36
37 # % formatting
38 message3 = "Hello, %s! You are %d years old." % (name, age)
39 print(f"% formatting: {message3}")
40
41 # String checking methods
42 email = "user@example.com"
43 print(f"Is email alphanumeric? {email.isalnum()}")
44 print(f"Is email lowercase? {email.islower()}")
45 print(f"Does email start with 'user'? {email.startswith('user')}")
46 print(f"Does email end with '.com'? {email.endswith('.com')}")
47
48 # String replacement
49 text = "I love Python programming"
50 new_text = text.replace("Python", "Java")
51 print(f"Original: {text}")
52 print(f"Replaced: {new_text}")
```

Original: Hello, World!
Uppercase: HELLO, WORLD!
Lowercase: hello, world!
Title case: Hello, World!
Length: 13
Original: ' Python is awesome! '
Stripped: 'Python is awesome!'
Left stripped: 'Python is awesome! '
Right stripped: ' Python is awesome!'
Split by comma: ['apple', 'banana', 'orange', 'grape']
Joined with ' | ': apple | banana | orange | grape
f-string: Hello, Alice! You are 30 years old.
.format(): Hello, Alice! You are 30 years old.
% formatting: Hello, Alice! You are 30 years old.
Is email alphanumeric? False
Is email lowercase? True
Does email start with 'user'? True
Does email end with '.com'? True
Original: I love Python programming
Replaced: I love Java programming

4. List Comprehensions and Generator Expressions

In [4...

```
1      # List comprehensions
2      numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
3
4
5      # Basic list comprehension
6      squares = [x**2 for x in numbers]
7      print(f"Squares: {squares}")
8
9      # List comprehension with condition
10     even_squares = [x**2 for x in numbers if x % 2 == 0]
11     print(f"Even squares: {even_squares}")
12
13     # List comprehension with multiple conditions
14     filtered = [x for x in numbers if x > 5 and x % 2 == 0]
15     print(f"Numbers > 5 and even: {filtered}")
16
17     # List comprehension with if-else
18     categorized = ["even" if x % 2 == 0 else "odd" for x in numbers]
19     print(f"Categorized: {categorized}")
20
21     # Nested list comprehension
22     matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
23     flattened = [item for row in matrix for item in row]
24     print(f"Flattened matrix: {flattened}")
25
26     # Dictionary comprehension
27     squares_dict = {x: x**2 for x in range(1, 6)}
28     print(f"Squares dictionary: {squares_dict}")
29
30     # Set comprehension
31     unique_lengths = {len(word) for word in ["apple", "banana", "grape"]}
32     print(f"Unique word lengths: {unique_lengths}")
33
34     # Generator expressions (memory efficient)
35     squares_gen = (x**2 for x in numbers)
```

```

38     print(f"Generator: {squares_gen}")
39     print(f"First 5 squares: {list(squares_gen)[:5]}")
40
41     # Generator with condition
    even_squares_gen = (x**2 for x in numbers if x % 2 == 0)
    print(f"Even squares generator: {list(even_squares_gen)}")

```

Squares: [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]

Even squares: [4, 16, 36, 64, 100]

Numbers > 5 and even: [6, 8, 10]

Categorized: ['odd', 'even', 'odd', 'even', 'odd', 'even', 'odd', 'even', 'odd', 'even']

Flattened matrix: [1, 2, 3, 4, 5, 6, 7, 8, 9]

Squares dictionary: {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

Unique word lengths: {4, 5, 6}

Generator: <generator object <genexpr> at 0x1113d53c0>

First 5 squares: [1, 4, 9, 16, 25]

Even squares generator: [4, 16, 36, 64, 100]

5. Object-Oriented Programming Basics

In [5...

```

1     # Basic class definition
2     class Person:
3         # Class variable
4         species = "Homo sapiens"
5
6
7     def __init__(self, name, age):
8         # Instance variables
9         self.name = name
10        self.age = age
11
12
13    def greet(self):
14        return f"Hello, I'm {self.name} and I'm {self.age} years
15
16    def have_birthday(self):
17        self.age += 1
18        return f"Happy birthday! Now I'm {self.age} years old."
19
20    # Creating objects
21    person1 = Person("Alice", 25)
22    person2 = Person("Bob", 30)
23
24    print(person1.greet())
25    print(person2.greet())
26
27    print(person1.have_birthday())
28    print(f"Species: {Person.species}")
29
30    # Accessing attributes
31    print(f"Person1 name: {person1.name}")
32    print(f"Person1 age: {person1.age}")
33
34    # Modifying attributes
35    person1.age = 26
    print(f"Updated age: {person1.age}")

```

Hello, I'm Alice and I'm 25 years old.
Hello, I'm Bob and I'm 30 years old.
Happy birthday! Now I'm 26 years old.
Species: Homo sapiens
Person1 name: Alice
Person1 age: 26
Updated age: 26

6. Inheritance and Polymorphism

In [6...

```
1  # Base class
2  class Animal:
3      def __init__(self, name, species):
4          self.name = name
5          self.species = species
6
7
8      def make_sound(self):
9          return "Some generic animal sound"
10
11     def move(self):
12         return f"{self.name} moves around"
13
14 # Derived class (inheritance)
15 class Dog(Animal):
16     def __init__(self, name, breed):
17         super().__init__(name, "Canis lupus")
18         self.breed = breed
19
20
21     def make_sound(self): # Method overriding
22         return "Woof! Woof!"
23
24     def fetch(self):
25         return f"{self.name} fetches the ball"
26
27 # Another derived class
28 class Cat(Animal):
29     def __init__(self, name, color):
30         super().__init__(name, "Felis catus")
31         self.color = color
32
33
34     def make_sound(self): # Method overriding
35         return "Meow! Meow!"
36
37     def climb(self):
38         return f"{self.name} climbs the tree"
39
40 # Creating objects
41 dog = Dog("Buddy", "Golden Retriever")
42 cat = Cat("Whiskers", "Orange")
43
44 # Polymorphism in action
45 animals = [dog, cat]
46
47 for animal in animals:
48     print(f"{animal.name} ({animal.species}): {animal.make_sound()}")
49     print(f"{animal.move()}")
50     print()
```

```
51
52     # Accessing specific methods
    print(dog.fetch())
    print(cat.climb())
```

Buddy (Canis lupus): Woof! Woof!
Buddy moves around

Whiskers (Felis catus): Meow! Meow!
Whiskers moves around

Buddy fetches the ball
Whiskers climbs the tree

7. File Reading and Writing

In [7...

```
1
2     # Writing to a file
3     data = ["Hello, World!", "This is a test file.", "Python is awesome"]
4
5     # Write mode
6     with open("sample.txt", "w") as file:
7         for line in data:
8             file.write(line + "\n")
9     print("File written successfully!")
10
11    # Reading from a file
12    with open("sample.txt", "r") as file:
13        content = file.read()
14        print("File content:")
15        print(content)
16
17    # Reading line by line
18    print("\nReading line by line:")
19    with open("sample.txt", "r") as file:
20        for i, line in enumerate(file, 1):
21            print(f"Line {i}: {line.strip()}")
22
23
24    # Appending to a file
25    with open("sample.txt", "a") as file:
26        file.write("This line was appended.\n")
27    print("\nLine appended!")
28
29    # Reading the updated file
30    print("\nUpdated file content:")
31    with open("sample.txt", "r") as file:
32        print(file.read())
33
34
35    # Working with CSV-like data
36    import csv
37
38    # Writing CSV data
39    csv_data = [
40        ["Name", "Age", "City"],
41        ["Alice", "25", "New York"],
42        ["Bob", "30", "Los Angeles"],
43        ["Charlie", "35", "Chicago"]
```



```

44     ]
45
46     with open("people.csv", "w", newline="") as file:
47         writer = csv.writer(file)
48         writer.writerows(csv_data)
49     print("CSV file written!")
50
51     # Reading CSV data
52     print("\nCSV file content:")
53     with open("people.csv", "r") as file:
54         reader = csv.reader(file)
55         for row in reader:
56             print(row)

```

File written successfully!
File content:
Hello, World!
This is a test file.
Python is awesome!

Reading line by line:
Line 1: Hello, World!
Line 2: This is a test file.
Line 3: Python is awesome!

Line appended!

Updated file content:
Hello, World!
This is a test file.
Python is awesome!
This line was appended.

CSV file written!

CSV file content:
['Name', 'Age', 'City']
['Alice', '25', 'New York']
['Bob', '30', 'Los Angeles']
['Charlie', '35', 'Chicago']

8. Exception Handling

In [8...

```

1     # Basic exception handling
2     def safe_divide(a, b):
3         try:
4             result = a / b
5             return result
6         except ZeroDivisionError:
7             print("Error: Cannot divide by zero!")
8             return None
9         except TypeError:
10            print("Error: Invalid input types!")
11            return None
12
13
14

```

```

15 print(safe_divide(10, 2))
16 print(safe_divide(10, 0))
17 print(safe_divide(10, "2"))
18
19 # Multiple exception types
20 def process_list(lst, index):
21     try:
22         return lst[index]
23     except IndexError:
24         print(f"Error: Index {index} is out of range!")
25         return None
26     except TypeError:
27         print("Error: Invalid index type!")
28         return None
29     except Exception as e:
30         print(f"Unexpected error: {e}")
31         return None
32
33
34 numbers = [1, 2, 3, 4, 5]
35 print(process_list(numbers, 2))
36 print(process_list(numbers, 10))
37 print(process_list(numbers, "2"))
38
39 # try-except-else-finally
40 def process_file(filename):
41     file = None
42     try:
43         file = open(filename, 'r')
44         content = file.read()
45         print("File read successfully!")
46     except FileNotFoundError:
47         print(f"Error: File '{filename}' not found!")
48         return None
49     except PermissionError:
50         print(f"Error: Permission denied to read '{filename}'")
51         return None
52     except Exception as e:
53         print(f"Unexpected error: {e}")
54         return None
55     else:
56         print("No errors occurred during file reading")
57         return content
58     finally:
59         if file:
60             file.close()
61             print("File closed successfully")
62
63
64
65 # This will show the error handling
66 result = process_file("nonexistent.txt")
67
68 # Custom exceptions
69 class CustomError(Exception):
70     def __init__(self, message):
71         self.message = message
72         super().__init__(self.message)
73
74 def validate_age(age):
75     if age < 0:
76         raise CustomError("Age cannot be negative!")
77     if age > 150:

```

```

79         raise CustomError("Age seems unrealistic!")
80     return True
81
82     try:
83         validate_age(-5)
84     except CustomError as e:
85         print(f"Custom error: {e}")
86
87     try:
88         validate_age(25)
89         print("Age validation passed!")
90     except CustomError as e:
91         print(f"Custom error: {e}")

```

```

5.0
Error: Cannot divide by zero!
None
Error: Invalid input types!
None
3
Error: Index 10 is out of range!
None
Error: Invalid index type!
None
Error: File 'nonexistent.txt' not found!
Custom error: Age cannot be negative!
Age validation passed!

```

9. Regular Expressions

In [9...

```

1  import re
2
3
4  # Basic pattern matching
5  text = "The quick brown fox jumps over the lazy dog"
6  pattern = r"fox"
7  match = re.search(pattern, text)
8  if match:
9      print(f"Found '{match.group()}' at position {match.start()}")
10 else:
11     print("Pattern not found")
12
13 # Finding all matches
14 pattern = r"\b\w{4}\b" # 4-letter words
15 matches = re.findall(pattern, text)
16 print(f"4-letter words: {matches}")
17
18 # Email validation
19 email_pattern = r"^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}"
20 emails = ["user@example.com", "invalid.email", "test@domain.co.uk"]
21
22
23 for email in emails:
24     if re.match(email_pattern, email):
25         print(f"'{email}' is a valid email")
26     else:
27         print(f"'{email}' is not a valid email")
28
29

```

```

30 # Phone number pattern
31 phone_pattern = r"(\d{3})?[-.\s]?(\d{3})?[-.\s]?(\d{4})"
32 phone_text = "Call me at (555) 123-4567 or 555.123.4567 or 555-12
33 phone_numbers = re.findall(phone_pattern, phone_text)
34 print(f"Phone numbers found: {phone_numbers}")
35
36 # Substitution
37 text = "The price is $19.99 and $29.99"
38 new_text = re.sub(r"\$\d+\.\d{2}", "PRICE", text)
39 print(f"Original: {text}")
40 print(f"After substitution: {new_text}")
41
42 # Splitting with regex
43 text = "apple,banana;cherry:date"
44 fruits = re.split(r"[,;:]", text)
45 print(f"Split fruits: {fruits}")
46
47 # Groups and capturing
48 pattern = r"(\d{4})-(\d{2})-(\d{2})"
49 date_text = "Today is 2024-01-15"
50 match = re.search(pattern, date_text)
51 if match:
    year, month, day = match.groups()
    print(f"Year: {year}, Month: {month}, Day: {day}")

```

Found 'fox' at position 16

4-letter words: ['over', 'lazy']

'user@example.com' is a valid email

'invalid.email' is not a valid email

'test@domain.co.uk' is a valid email

'not-an-email' is not a valid email

Phone numbers found: ['(555) 123-4567', '555.123.4567', '555-123-4567']

Original: The price is \$19.99 and \$29.99

After substitution: The price is PRICE and PRICE

Split fruits: ['apple', 'banana', 'cherry', 'date']

Year: 2024, Month: 01, Day: 15

10. Working with Dates and Time

In [1...

```

1 from datetime import datetime, date, time, timedelta
2 import time as time_module
3
4 # Current date and time
5 now = datetime.now()
6 today = date.today()
7 current_time = time(now.hour, now.minute, now.second)
8
9
10 print(f"Current datetime: {now}")
11 print(f"Today's date: {today}")
12 print(f"Current time: {current_time}")
13
14 # Creating specific dates and times
15 birthday = date(1990, 5, 15)
16 appointment = datetime(2024, 3, 15, 14, 30, 0)
17
18 print(f"Birthday: {birthday}")
19

```

```

20 print(f"Appointment: {appointment}")
21
22 # Date arithmetic
23 age = today - birthday
24 print(f"Age in days: {age.days}")
25 print(f"Age in years: {age.days // 365}")
26
27 # Adding/subtracting time
28 future_date = today + timedelta(days=30)
29 past_date = today - timedelta(weeks=2)
30
31
32 print(f"30 days from now: {future_date}")
33 print(f"2 weeks ago: {past_date}")
34
35 # Formatting dates
36 formatted_date = now.strftime("%Y-%m-%d %H:%M:%S")
37 print(f"Formatted date: {formatted_date}")
38
39 # Parsing dates from strings
40 date_string = "2024-01-15 14:30:00"
41 parsed_date = datetime.strptime(date_string, "%Y-%m-%d %H:%M:%S")
42 print(f"Parsed date: {parsed_date}")
43
44 # Time zones (basic example)
45 print(f"Current timestamp: {now.timestamp()}")
46
47 # Sleep example (commented out to avoid delay)
48 # print("Sleeping for 2 seconds...")
49 # time_module.sleep(2)
50 # print("Awake!")
51
52
53 # Date comparison
54 date1 = date(2024, 1, 15)
55 date2 = date(2024, 1, 20)
56
57 if date1 < date2:
58     print(f"{date1} is before {date2}")
59 else:
60     print(f"{date1} is after {date2}")
61
62
63 # Working with weekdays
64 weekdays = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday"]
65 today_weekday = weekdays[today.weekday()]
66 print(f"Today is {today_weekday}")
67
68 # Days until next Monday
69 days_until_monday = (7 - today.weekday()) % 7
70 if days_until_monday == 0:
71     days_until_monday = 7
72 print(f"Days until next Monday: {days_until_monday}")

```

Current datetime: 2025-09-07 21:55:56.626558
Today's date: 2025-09-07
Current time: 21:55:56
Birthday: 1990-05-15
Appointment: 2024-03-15 14:30:00
Age in days: 12899
Age in years: 35
30 days from now: 2025-10-07
2 weeks ago: 2025-08-24
Formatted date: 2025-09-07 21:55:56
Parsed date: 2024-01-15 14:30:00
Current timestamp: 1757307356.626558
2024-01-15 is before 2024-01-20
Today is Sunday
Days until next Monday: 1

In [...

1
2